

-- RETAIL INDUSTRY PERFORMANCE & PROMOTION ANALYSIS

-- Database Engine: Microsoft SQL Server

```
--  
=====
```

-- 1. DATA CLEANING & NULL VALUE CHECK

```
SELECT *  
FROM fact_events  
WHERE event_id IS NULL  
      OR store_id IS NULL  
      OR campaign_id IS NULL  
      OR product_code IS NULL  
      OR base_price IS NULL  
      OR promo_type IS NULL  
      OR quantity_sold_before_promo IS NULL  
      OR quantity_sold_after_promo IS NULL;
```

-- 2. KEY PERFORMANCE METRICS (KPIs)

-- Total Incremental Revenue (IR)

```
SELECT  
    (SUM(CAST(base_price AS BIGINT) * CAST(quantity_sold_after_promo AS BIGINT)) -  
     SUM(CAST(base_price AS BIGINT) * CAST(quantity_sold_before_promo AS BIGINT))) AS  
    [Incremental Revenue (IR)]  
FROM fact_events;
```

-- Total Incremental Sold Units (ISU)

```
SELECT
```

```
(SUM(quantity_sold_after_promo) - SUM(quantity_sold_before_promo)) AS Incremental_Sold_Units
FROM fact_events;
```

-- 3. STORE PERFORMANCE ANALYSIS

```
-- Top 10 Stores by Incremental Revenue
```

```
WITH store_performance AS (
```

```
    SELECT
```

```
        f.store_id,
```

```
        SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_before_promo AS BIGINT)) AS
Sales_Before,
```

```
        SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_after_promo AS BIGINT)) AS
Sales_After
```

```
    FROM fact_events AS f
```

```
    GROUP BY f.store_id
```

```
),
```

```
store_incremental AS (
```

```
    SELECT *, (Sales_After - Sales_Before) AS Incremental_Revenue
```

```
    FROM store_performance
```

```
),
```

```
store_rank AS (
```

```
    SELECT *, ROW_NUMBER() OVER(ORDER BY Incremental_Revenue DESC) AS Rank
```

```
    FROM store_incremental
```

```
)
```

```
SELECT * FROM store_rank WHERE Rank <= 10 ORDER BY Rank;
```

```
-- Bottom 10 Stores by Incremental Sold Units
```

```
WITH store_performance AS (
```

```
    SELECT
```

```

        store_id,
        SUM(quantity_sold_before_promo) AS Units_Before,
        SUM(quantity_sold_after_promo) AS Units_After
    FROM fact_events
    GROUP BY store_id
),
store_incremental AS (
    SELECT *, (Units_After - Units_Before) AS Incremental_Sold_Units
    FROM store_performance
),
store_rank AS (
    SELECT *, ROW_NUMBER() OVER(ORDER BY Incremental_Sold_Units ASC) AS Rank
    FROM store_incremental
)
SELECT * FROM store_rank WHERE Rank <= 10 ORDER BY Rank;

```

-- 4. CITY & STORE RANKING ANALYSIS

```

-----

-- City-wise Incremental Revenue Performance & Ranking
WITH cte1 AS (
    SELECT
        s.city,
        SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_before_promo AS BIGINT)) AS
        Total_sales_before_promo,
        SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_after_promo AS BIGINT)) AS
        Total_sales_after_promo
    FROM fact_events AS f
    LEFT JOIN dim_stores AS s ON f.store_id = s.store_id
    GROUP BY s.city
),

```

```

cte2 AS (
    SELECT *, (Total_sales_after_promo - Total_sales_before_promo) AS Incremental_Revenue
    FROM cte1
)
SELECT *, DENSE_RANK() OVER(ORDER BY Incremental_Revenue DESC) AS City_Rank
FROM cte2;

```

-- Top 10 Stores Overall by Incremental Revenue (with City Info)

```

WITH cte1 AS (
    SELECT
        s.city,
        f.store_id,
        SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_before_promo AS BIGINT)) AS
Total_sales_before_promo,
        SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_after_promo AS BIGINT)) AS
Total_sales_after_promo
    FROM fact_events AS f
    LEFT JOIN dim_stores AS s ON f.store_id = s.store_id
    GROUP BY s.city, f.store_id
),
cte2 AS (
    SELECT *, (Total_sales_after_promo - Total_sales_before_promo) AS Incremental_Revenue
    FROM cte1
),
cte3 AS (
    SELECT *, DENSE_RANK() OVER(ORDER BY Incremental_Revenue DESC) AS Rank
    FROM cte2
)
SELECT * FROM cte3 WHERE Rank <= 10 ORDER BY Rank;

```

-- 5. CATEGORY & PROMOTION EFFECTIVENESS

-- Product Category-Wise Incremental Sold Units

```
SELECT
    p.category,
    SUM(f.quantity_sold_before_promo) AS Total_Units_Before_Promo,
    SUM(f.quantity_sold_after_promo) AS Total_Units_After_Promo,
    (SUM(f.quantity_sold_after_promo) - SUM(f.quantity_sold_before_promo)) AS
Incremental_Sold_Units
FROM fact_events AS f
LEFT JOIN dim_products AS p ON f.product_code = p.product_code
GROUP BY p.category;
```

-- Product Category x Promotion Type Incremental Revenue

```
SELECT
    p.category,
    f.promo_type,
    (SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_after_promo AS BIGINT)) -
    SUM(CAST(f.base_price AS BIGINT) * CAST(f.quantity_sold_before_promo AS BIGINT))) AS
Incremental_Revenue
FROM fact_events AS f
LEFT JOIN dim_products AS p ON f.product_code = p.product_code
GROUP BY p.category, f.promo_type;
```

-- Promotion Type Performance with Contribution Percentage (%)

WITH cte1 AS (

```
    SELECT
        promo_type,
```

```

        (SUM(quantity_sold_after_promo) - SUM(quantity_sold_before_promo)) AS
Incremental_Sold_Units,

        (SUM(CAST(base_price AS BIGINT) * CAST(quantity_sold_after_promo AS BIGINT)) -
        SUM(CAST(base_price AS BIGINT) * CAST(quantity_sold_before_promo AS BIGINT))) AS
Incremental_Revenue

FROM fact_events

GROUP BY promo_type
),

cte2 AS (

    SELECT

        (SUM(CAST(base_price AS BIGINT) * CAST(quantity_sold_after_promo AS BIGINT)) -
        SUM(CAST(base_price AS BIGINT) * CAST(quantity_sold_before_promo AS BIGINT))) AS
total_Incremental_Revenue,

        (SUM(quantity_sold_after_promo) - SUM(quantity_sold_before_promo)) AS
total_Incremental_sold_Units

    FROM fact_events

)

SELECT

    cte1.promo_type,

    cte1.Incremental_Sold_Units,

    CAST((CAST(cte1.Incremental_Sold_Units AS DECIMAL(18,4)) /
cte2.total_Incremental_sold_Units) * 100 AS DECIMAL(10,2)) AS Sold_Units_Contribution_Pct,

    cte1.Incremental_Revenue,

    CAST((CAST(cte1.Incremental_Revenue AS DECIMAL(18,4)) / cte2.total_Incremental_Revenue) *
100 AS DECIMAL(10,2)) AS Revenue_Contribution_Pct

FROM cte2

CROSS JOIN cte1;

```